



I'm trying to set up a kind of "three-layer explanation system" for understanding the Muon optimizer from Lecture 7. I learn better when I can see both the intuitive story and the mathematical machinery, and also some kind of "middle" explanation that interprets the math in terms of geometry or structure (not just raw formulas).

So whenever I ask you questions about Muon, can you answer using the three layers below? I'm writing this out in some detail because I want the structure to be consistent across questions.

(A) Why I want three layers (just so you know how to respond)

In my course, EECS 182 at UC Berkeley, we generally can't get away with just knowing formulas — we have to actually understand what the update rule is doing and why it is shaped that way. And the opposite is also true: intuition without derivation isn't enough to survive the homeworks or really understand the lecture.

So the three layers I want are basically:

- Layer 1: Intuitive explanation

What problem Muon solves, why direction matters, why semi-orthogonalization helps, etc.

- Layer 2: "Geometry/structure" explanation

Not formal derivation, but not metaphor either — something in between, like how the SVD is shaping the gradient, what replacing Σ with I means, how skewed directions get "evened out," what Newton–Schulz does to singular values, etc.

This is where you can use notation, SVD identities, norm constraints, derivations, etc.

Try not to skip key steps, because these "missing steps" are usually what confuse me.

Just giving these three consistently will help me compare the perspectives and build a clearer mental model.

(B) The actual instructions for each layer

Layer 1 — Intuitive (no formulas necessary)

Explain the idea like you're talking to a classmate who missed the lecture.

Focus on:

- what problem the naive gradient step has,
- why we would want the "direction" of the update independent of its distortions,
- why Muon cares about removing the singular values,
- why approximate orthogonalization is "good enough."

This level should be friendly and concept-shaped, not math-shaped.

If you want a really hand-wavy way of describing what this layer is, imagine you're a student who attended lecture, and I am not. I ask you briefly when I pass you in the hallway, "What did I miss in lecture? What did it cover?" And you answer something that starts with "Basically..." You should aim for the BASICALLY here — get the point

understand it verbally first.

Layer 2 — Geometry / structural intuition

Explain the shapes, flows, and transformations:

- what UV^T represents geometrically,
- how singular values stretch directions,
- why controlling direction instead of magnitude gives more stable steps,
- how the Newton–Schulz polynomial gradually "pulls" singular values toward 1.

This is basically the "bridge" between intuition and derivation. No need to be super formal, but this layer should honor actual linear algebra behavior.

Layer 3 — Mathematical derivation / formal view

Here you can walk through things like:

- the constrained optimization viewpoint (minimize $\langle G, \Delta W \rangle$ with norm constraints)

I intentionally spent time writing a long setup prompt here. The goal was to force the model into a fixed three-layer pattern (intuition → geometry → math) so that every later answer would be structurally comparable. This ended up making it much easier to see where the model was strong or weak at different levels, in addition to the inherent pedagogical value of explaining things in different ways.

norm constraints),

- why the solution involves the top singular vectors of the gradient,
- how we get from $G = U \Sigma V^T$ to wanting UV^T ,
- why Newton–Schulz works as an approximation of the orthogonal factor

↓

- what assumptions or normalizations are required.

Whenever you ask a question about Muon (or related concepts in EECS 182), I will answer in **three consistent layers** so I can follow the logic.

(C) General behavior notes

Here are some things that help me learn:

1. Stick to the three layers unless I explicitly say otherwise.
2. It's okay to be a bit verbose — I'm using this to study, not to get quick answers.
3. If something is uncertain or depends on conditions (like convergence of Newton–Schulz), just say so. Don't assume that I know. Remember, I am a student who wants to LEARN.
4. If I ask a follow-up, you don't need to repeat absolutely everything, but still use the three-layer structure.
5. Occasionally, you can ask me a small comprehension question if it naturally fits.

While this is a long set of instructions, it is highly important that you remember them as the student studies with you. I want to set up a reusable structure for clarifying Muon and related concepts. Let's start.

Great — I understand the structure you want, and I can absolutely follow it for all future questions about Muon or related topics. I'll summarize back to you v ↓ I've internalized, then confirm I'm ready.

✓ My understanding of your instructions

Whenever you ask a question about Muon (or related concepts in EECS 182), I will answer in **three consistent layers**:

Layer 1 — Intuitive explanation (“Basically...”)

- No formulas needed.
- Friendly, verbal, concept-level.
- As if a student is telling another student what the lecture *was basically about*.
- Focus on *why* the procedure exists and what problem it solves.

Layer 2 — Geometry / structure

- Uses correct linear algebra ideas but not formal derivations.
- Describes shapes, flows, stretching, orthogonal factors, etc.
- Explains how things like SVD components, singular values, Newton–Schulz iterations behave.
- Bridges intuition ↔ math.

Layer 3 — Mathematical explanation

- Formal but clear derivations.
- Symbolic steps should not skip key transitions. ↓
- Covers constrained optimization view. SVD identities. orthogonalization approximations, etc.

I, too, will list assumptions and conditions (e.g., internalizing, convergence).

It was nice to see the model restate my instructions back to me. This confirmed that it actually internalized the three-layer structure instead of just ignoring the long prompt, which is something that often happens with less constrained prompting.

Additional rules I will follow

- I will *always* keep the three-layer structure unless you say otherwise.
- I can be verbose since this is for studying.
- I will clarify uncertainties (e.g., convergence assumptions).
- When you follow up, I will still use the three-layer format (but I won't repeat irrelevant background).
- I can occasionally ask small comprehension questions if appropriate.

I'm ready whenever you are

You can now ask your **first actual question about Muon**, and I will answer using the three layers exactly as specified.

What would you like to explore first?

📄 🍌 🗨️ ⬆️ ↺️ ⋮

To get started, could you walk me through Muon from the beginning using the three-layer format? I'm mainly trying to get a clear "full picture" version I can refer to when asking more detailed questions, so feel free to take a bit of space. I don't need anything hyper-polished; I just want

views line up with each other.

Below is a **full-picture walkthrough** of Muon, presented in the **three-layer structure** you requested. This is intentionally a bit expansive so you can "see the whole landscape" before drilling into details.

Layer 1 — Intuitive explanation ("Basically...")

Basically, Muon is an optimizer that tries to fix the distortions that show up in standard gradient descent when your weight matrices have very uneven scaling along different directions.

Here's the story in simple terms:

1. The problem with normal gradient descent

If a weight matrix W has some directions in which it stretches space a lot and others where it barely does anything, then the gradient you compute will reflect those distortions.

So instead of telling you the "true direction of improvement," the gradient is warped by the geometry of W itself.

This leads to:

- updates that are too large in some directions,
- too small in others,
- and overall unstable or slow training.



2. What Muon wants

Muon tries to update w in a direction that depends only on the meaningful shape of the gradient, not on how W happens to be skewed.

To do this, Muon tries to **remove the distortion introduced by the singular values of G** (your gradient).

The singular vectors tell you *where* the matrix points; the singular values tell you *how strongly* it pushes in each direction.

Muon says:

"I only care about the directions, not the accidentally over- or under-weighted magnitudes."

So it tries to use only the **directional part** of the gradient.

3. So what does Muon actually do?

It tries to approximate the **orthogonal factor** of the gradient's SVD — basically the "pure directional component" of the gradient.

But computing a full SVD every step is too expensive, so instead it uses an iterative trick (Newton–Schulz) that gradually **pushes the gradient toward something whose singular values are all 1**, meaning it keeps the directions but removes the distortions.

4. Why approximate orthogonalization is okay

You don't need perfect orthogonalization — just enough to ensure the update direction isn't being dominated by weird scaling.

Newton–Schulz converges quickly, so a few steps get you "close enough."

This stabilizes training and makes updates more isotropic, leading to smoother optimization.

Muon takes the gradient, scrubs off the uneven stretching, keeps the directional information, and uses that cleaned-up version to update the weights.

I like how it frames Muon as correcting "shape" rather than just shrinking numbers. That aligns with how the lecture motivated Muon via badly conditioned gradients. It's a good high-level story that matches my own mental picture.

Layer 2 — Geometry / structural intuition

Now let's describe the linear-algebra shapes involved.

1. SVD as stretching-and-rotating

If the gradient is G , its SVD is:

$$G = U\Sigma V^T.$$

Geometrically:

- V^T rotates your input space,
- Σ stretches or squashes different axes,
- U rotates again.

All distortions of magnitude live in Σ .

The “pure directionality” — the transformation that doesn’t distort lengths — is given by:

$$Q = UV^T \quad (\text{orthogonal matrix}).$$

This **preserves lengths** and only rotates.



So geometrically, Muon tries to replace

“rotate + stretch + rotate”
with
“rotate → rotate”
(i.e., no stretch).

2. Why this matters for optimization

The gradient is supposed to tell you *which way to move* to reduce loss.

But if some axes are stretched 100x more than others, then your gradient’s direction is dominated by those axes even if they’re not semantically important.

This is like trying to walk downhill when your map is warped — the map still points downhill, but the stretching makes the steepness misleading.

3. What Newton–Schulz does geometrically

Newton–Schulz is an iterative method that transforms a matrix G into something close to an orthogonal matrix.

The iteration has the rough shape:

$$X_{k+1} = \frac{1}{2}X_k(3I - X_k^T X_k)$$

(ignoring normalization factors for now).



How does this null singular values toward 1?

If you look at a single singular value σ , the scalar version of the iteration behaves like:

$$\sigma_{k+1} = \frac{1}{2}\sigma(3 - \sigma^2).$$

Graphically, this map:

- pulls values greater than 1 downward toward 1,
- pulls values less than 1 upward toward 1,
- fixes 1 as a stable point.

So geometrically the iteration **pushes the stretched ellipsoid back into a sphere**.

End result

After a few iterations:

- directions (U and V) remain almost unchanged,
- singular values become nearly all 1,
- giving a good approximation to the orthogonal factor UV^T .

This is the “clean” gradient direction Muon uses.

4. Update step geometry

If the update is $\Delta W = -\eta \tilde{G}$ where $\tilde{G} \approx UV^T$:

- all directions are treated evenly,
- step sizes are chosen independently of arbitrary scaling,
- optimization becomes more stable and follows the intrinsic geometry of the problem.

This is a clean geometric explanation of why UV^T is the “pure direction” update. One thing it doesn’t mention is that in Muon we never literally compute UV^T ; we only approximate it with polynomial iterations, which is important for understanding the computational side.

Layer 3 — Mathematical derivation / formal view

Now the formal reasoning that leads to the Muon update.

1. Directional update under norm constraint

A classical way to derive the Muon goal is:

Minimize $\langle G, \Delta W \rangle$
subject to $\|\Delta W\|_F = c$
(i.e., take the steepest descent direction with a fixed-size step).

This reduces to finding the matrix ΔW of fixed norm that has the largest negative inner product with G .

By Cauchy–Schwarz:

$$\langle G, \Delta W \rangle \geq -\|G\|_F \|\Delta W\|_F.$$

Equality happens when

$$\Delta W \propto -G.$$

But this is not what Muon wants — it's what plain GD does.

Muon adds an important constraint:

We want the update to be **isotropic** in the sense that its effective action doesn't depend on the singular values.



Minimize $\langle G, \Delta W \rangle$
subject to $\Delta W^T \Delta W = I$ (orthogonality constraint).

This yields a very different solution.

2. Solving with the SVD

Let $G = U\Sigma V^T$.

We want:

$$\Delta W = \arg \min_{\Delta W^T \Delta W = I} \langle G, \Delta W \rangle.$$

Note:

$$\langle G, \Delta W \rangle = \text{Tr}(G^T \Delta W) = \text{Tr}(V \Sigma U^T \Delta W).$$

Let $X = U^T \Delta W V$.

Because ΔW is orthogonal, X is also orthogonal.

Then:

$$\langle G, \Delta W \rangle = \text{Tr}(\Sigma X).$$

Because Σ is diagonal with nonnegative entries, the trace is minimized when:

$$X \downarrow -I.$$

$U^T \Delta W V = -I,$

which gives:

$$\Delta W = -UV^T.$$

This is **exactly the orthogonal factor** of the SVD.

3. Why not compute UV^T explicitly?

A full SVD per layer per iteration would be far too slow.

So Muon tries to approximate:

$$UV^T \approx G(G^T G)^{-1/2}.$$

This identity comes from:

$$G = U\Sigma V^T \Rightarrow G^T G = V\Sigma^2 V^T \Rightarrow (G^T G)^{-1/2} = V\Sigma^{-1} V^T.$$

Then:

$$G(G^T G)^{-1/2} = U\Sigma V^T \cdot V\Sigma^{-1} V^T = UIV^T = UV^T.$$

So orthogonalization reduces to computing a matrix inverse square root.



4. Newton–Schulz as the approximate inverse square root

Newton–Schulz uses an iterative scheme:

$$X_{k+1} = \frac{1}{2} X_k (3I - X_k^T X_k),$$

The derivation here is basically right, but it skips a detail from Lecture 7 where we switch from spectral norm to an RMS-style norm to get a more reasonable scaling. The model talks about a generic norm constraint and doesn't fully reflect that nuance.

with appropriate normalization of X_0 .

It converges (when suitably scaled) to:

$$X_\infty = (G^T G)^{-1/2}.$$

So the Muon update becomes:

$$\Delta W \approx -\eta G X_k.$$

After a few iterations, $G X_k \approx UV^T$, giving a clean directional update.

The description here mixes together two related uses of Newton–Schulz (for inverse square roots and for orthogonalization). In Muon we care specifically about driving singular values toward 1 to approximate UV^T , not about computing a precise matrix inverse. The high-level idea is fine, but the wording is a bit conflated from my understanding after lecture.

5. Assumptions

- $G^T G$ must be well-conditioned enough for Newton–Schulz to converge (often ensured via normalization).
- A few iterations suffice in practice — exact orthogonality is not required.
- This framework treats the gradient as acting in matrix space, which is standard in deep learning layers with weight matrices.



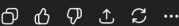
Summary across all layers

Layer	Core idea
Intuition	Muon removes distortion from the gradient so updates follow the true direction and not the warped magnitudes.
Geometry	SVD describes rotation → stretch → rotation; Muon removes the stretch (singular values), keeping only the directional part. Newton–Schulz collapses all singular values to 1.
Math	The constrained minimization under an orthogonality condition yields update $-UV^T$. Newton–Schulz approximates $G(G^T G)^{-1/2} = UV^T$ efficiently.

If you want, your next question could be about:

- Newton–Schulz convergence,
- why removing singular values helps generalization,
- how Muon compares to weight normalization,
- or walking through one example numerically.

What part of Muon do you want to zoom into next?



Thanks, that helps. I think the part I'm still fuzzy on is the Newton–Schulz part. Could you go through that again a bit more slowly using the same three-layer structure? I get the basic idea of the polynomial,

values toward 1, or what exactly goes wrong if the matrix isn't normalized properly first. I'm also curious about how people think about this step conceptually—like what it is “doing” to the matrix at each iteration.

Below is a slower, more detailed, **three-layer walkthrough of Newton–Schulz**, with special attention to:

- why the iteration drives singular values toward 1,
- why normalization matters,
- and the *conceptual picture* of what each iteration “does” to the matrix.

Layer 1 — Intuitive explanation (“Basically...”)

Basically, Newton–Schulz is a fast way to make a matrix “more orthogonal” by repeatedly correcting how much it stretches space.

Here's the idea in plain language:

1. What we want

We want a matrix whose singular values are all 1.

That means:

- no stretching,
- no squashing,
- just pure rotation/reflection.



2. What Newton–Schulz does

Newton–Schulz repeatedly *nudges* a matrix toward orthogonality by looking at how far it is from being orthogonal.

A matrix X is perfectly orthogonal when:

$$X^T X = I.$$

So Newton–Schulz applies an update that tries to fix that error.

If $X^T X$ is too big, it shrinks the matrix;

if $X^T X$ is too small, it expands it.

3. Why it works

The update formula has a built-in way of pulling singular values toward 1:

- if a singular value is too large, the iteration makes it smaller;
- if it's too small, the iteration makes it bigger.

So Newton–Schulz is like a “singular-value equalizer” that rounds all of them toward 1.

4. Why normalization is needed

If the matrix is *too distorted* (singular values way too far from 1), the correction step can overshoot — instead of pulling values toward 1, it may push them further away.

Normalization keeps the eigenvalues in a stable region where the iteration works reliably.

5. Conceptually



Each iteration is taking your current matrix, checking how much it deviates from being length-preserving, and then adjusting it.

You can think of it like:

“Rotate/stretch → measure error → apply a correction → rotate/stretch → measure error → ...”
until the stretching disappears.

The model underplays how important the normalization step is. In class, we were told that you really want the singular values scaled into (0,1) before applying the polynomial; otherwise the iteration can misbehave. While the model does mention these points in different words, it places less emphasis on the concept.

Layer 2 — Geometry / Structural Explanation

Let's zoom in on the geometric shape of Newton–Schulz.

1. What are we iterating toward?

We want something like:

$$Q = A(A^T A)^{-1/2},$$

which has the same singular vectors as A but singular values all equal to 1.

Geometrically:

- A maps the unit sphere to an ellipsoid.
- An orthogonal matrix maps the unit sphere to the sphere again.
- Newton–Schulz tries to **turn the ellipsoid back into a sphere**.



2. How Newton–Schulz changes singular values

Take the Newton–Schulz update and simplify it by scaling:

$$X_{k+1} = \frac{1}{2} X_k (3I - X_k^T X_k).$$

Let σ be a singular value of X_k .

Because the singular vectors don't change, the action on singular values mirrors the scalar update:

$$\sigma_{k+1} = \frac{1}{2} \sigma (3 - \sigma^2).$$

This is a nonlinear map with the following shape:

- If $\sigma > 1$, then $\sigma_{k+1} < \sigma$;
- If $\sigma < 1$, then $\sigma_{k+1} > \sigma$;
- $\sigma = 1$ is a stable equilibrium.

Geometrically, this is exactly the “ellipsoid-to-sphere” correction.

You can imagine each singular axis being pulled toward radius = 1.

3. Why normalization matters (geometric picture)

Newton–Schulz behaves well if all singular values start reasonably close to 1 — typically within (0, 2).

If A has singular values that are, say, 100 or 0.001, then:

- the update for those singular values can explode,
- the transformation can overshoot or diverge,

- the matrix may get pushed into a region where $\downarrow$ iteration no longer contracts.

So before running the iteration, we scale:

$$A_0 = \frac{A}{\alpha},$$

where α is chosen to constrain singular values into the convergence interval.

This ensures the “ellipsoid” is not too extreme before you start the “shrink-to-sphere” process.

4. Conceptual picture of what happens each iteration

Each iteration does something like:

1. Measure distortion:

Compute $X_k^T X_k$.

This tells how stretched the image of the unit sphere is (covariance of mapped unit vectors).

2. Design correction:

Form the polynomial $3I - X_k^T X_k$.

This says:

- if eigenvalue > 1 , shrink;
- if eigenvalue < 1 , expand.

3. Apply correction:

Multiply by X_k to update the map.

4. Repeat until the ellipsoid is nearly a sphere.

Geometrically, Newton–Schulz treats the singular directions independently but keeps the orientations fixed.



Layer 3 — Mathematical Derivation and Formal Reasoning

Now for the formal math behind convergence and polynomial behavior.

1. What problem Newton–Schulz solves

We want to compute:

$$A^{-1/2} = (A^T A)^{-1/2}.$$

One classical approach is to find a fixed point of the function:

$$f(X) = \frac{1}{2}X(3I - X^T X).$$

At a fixed point:

$$X = \frac{1}{2}X(3I - X^T X) \quad \Rightarrow \quad 2I = 3I - X^T X \quad \Rightarrow \quad X^T X = I.$$

Thus any orthogonal matrix is a fixed point.

When starting from $X_0 = A/\alpha$, the iteration converges to $A^{-1/2}$ (up to scaling) if singular values are in the correct region.

2. Why singular values evolve independently

Let the SVD of X_k be:

$$X_k = \begin{matrix} \downarrow \\ U \end{matrix} \Sigma_k V^T.$$

$$X_k^T X_k = V \Sigma_k^2 V^T.$$

So:

$$3I - X_k^T X_k = V(3I - \Sigma_k^2)V^T.$$

Thus:

$$X_{k+1} = \frac{1}{2}U\Sigma_k V^T V(3I - \Sigma_k^2)V^T = U\left(\frac{1}{2}\Sigma_k(3I - \Sigma_k^2)\right)V^T.$$

Hence the singular vectors stay the same; only the singular values change via:

$$\sigma_{k+1} = \frac{1}{2}\sigma_k(3 - \sigma_k^2).$$

This is a polynomial map of degree 3.

3. Why the polynomial pulls values toward 1

Consider the scalar function:

$$g(\sigma) = \frac{1}{2}\sigma(3 - \sigma^2).$$

We check fixed points:

$$g(\sigma) = \sigma \quad \Rightarrow \quad \frac{1}{2}\sigma(3 - \sigma^2) = \sigma.$$

Assuming $\sigma \neq 0$:



$$3 - \sigma^2 = 2 \quad \Rightarrow \quad \sigma^2 = 1.$$

Thus $\sigma = \pm 1$.

(We only care about $\sigma > 0$, so we choose $\sigma = 1$.)

Next, check stability by derivative:

$$g'(\sigma) = \frac{1}{2}(3 - \sigma^2) - \sigma^2 = \frac{3}{2} - \frac{3}{2}\sigma^2.$$

At $\sigma = 1$:

$$g'(1) = \frac{3}{2} - \frac{3}{2} = 0.$$

A derivative of 0 means **quadratic convergence** toward 1 — extremely fast.

Away from 1:

- If $0 < \sigma < \sqrt{3}$, then $g'(\sigma)$ is positive and < 1 .
- If $\sigma > \sqrt{3}$, the map turns unstable (hence normalization is needed).

This is why we scale A so that all singular values lie in $(0, 2)$, preferably narrower.

4. Role of normalization (formal)

If A has SVD $U\Sigma V^T$, we scale:

$$X_0 = \frac{A}{\alpha},$$

where $\alpha \approx \|A\|_2$ or some bound on the largest singular value.



Then new singular values satisfy

$$\sigma_0 = \frac{\sigma(A)}{\alpha} \leq 1.$$

This places all singular values below 1, guaranteeing Newton–Schulz is contractive.

After computing an approximate inverse square root, you rescale appropriately to undo the normalization.

5. What the iteration approximates

We know:

$$A^{-1/2} = \lim_{k \rightarrow \infty} X_k,$$

and hence:

$$A \cdot A^{-1/2} = A^{1/2}.$$

Muon then uses:

$$AA^{-1/2} \approx U\Sigma V^T \cdot V\Sigma^{-1}V^T = UV^T.$$

Thus Newton–Schulz gives a practical way to obtain the orthogonal factor UV^T without explicit SVD.

Summary Across Layers



Layer	Main insight
Intuitive	Newton–Schulz repeatedly corrects the matrix to make its singular values closer to 1 — stable, quadratic convergence in preserving spectrum.
Geometric	Each singular direction is kept the same, but its stretching factor is pushed toward radius 1, turning an ellipsoid into a sphere.
Mathematical	The iteration produces a polynomial update on singular values: $\sigma_{k+1} = \frac{1}{2}\sigma(3 - \sigma^2)$, which has 1 as a stable fixed point, and normalization ensures convergence.

If you want, we can next explore:

- why convergence is quadratic,
- the relationship to matrix square roots,
- how many iterations Muon typically uses,
- or even step through a numerical example.

What would you like to look at next?

Alright, this is making more sense now. Before we move on, could you help me connect this to other optimizers a bit more concretely? I know Muon is trying to fix some conditioning issues that Adam or SGD don't really address, but I'd like to hear that comparison in the same three-layer $\downarrow$ 'e. I'm especially interested in what the update actually looks like relative to something like Adam or even Shampoo.

choice, feel free to mention those too.

Below is a **three-layer comparison** explaining how **Muon** relates to **SGD**, **Adam**, and **Shampoo**, and *when Muon might actually be the wrong tool*.
As before, each layer gets progressively more technical.

Layer 1 — Intuitive Explanation (“Basically...”)

Basically, Muon tries to fix a problem that SGD and Adam don't even attempt to solve: the fact that the gradient's *direction* is distorted by the weight matrix's internal geometry.

1. SGD

SGD just says:

“Take a step in the direction of the gradient.”

This works fine if the landscape is round-ish, but if the geometry is skewed (some directions really steep, some barely sloped) SGD becomes slow or unstable.

2. Adam

Adam says:

“I'll adapt the *magnitude* of the step in each coordinate so none of them blow up.”

So if the gradient itself is badly distorted (because the layer is ill-conditioned), Adam can't fully fix it.

Adam cares about **per-parameter scaling**, not about **matrix-level geometry**.

3. Shampoo

Shampoo says:

“Let's precondition gradients using second-order (approximate curvature) information at the matrix level.”

Shampoo does fix conditioning more intelligently than Adam, but it requires:

- computing and storing large preconditioners,
- maintaining Kronecker or matrix statistics,
- expensive matrix inverse roots.

It improves conditioning but **does not explicitly orthogonalize** the gradient.

4. Muon

Muon says:

“Let's clean up the *direction* of the gradient, removing distortions caused by singular values, so optimization follows the true geometry.”

Muon does not rescale each coordinate like Adam, and it does not build second-order statistics like Shampoo.

Instead, it produces an update that behaves like **SGD on a well-conditioned matrix**, regardless of how skewed the actual gradient is.

5. When Muon can be a bad choice

- If your layer isn't a matrix (e.g., convolution kernels reshaped awkwardly), applying Muon may add unnecessary overhead.
- If the gradient is extremely noisy (tiny batch sizes), the orthogonal correction may amplify noise.
- If your model is tiny, Muon overhead may dominate.
- If your gradients are already well-conditioned (e.g., certain normalized architectures), Muon gives limited benefit.

Conceptually:

Muon is a “directional cleaner,” not a magnitude smoother (Adam) and not a curvature estimator (Shampoo). So if your problem doesn’t need direction-cleaning, Muon isn’t particularly helpful.

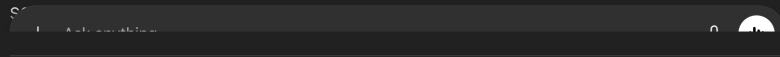
Layer 2 — Geometry / Structural Comparison

1. Geometry of SGD

SGD uses $-G$.

Geometrically:

- It follows the gradient field directly.
- If the gradient lives in a long, cigar-shaped valley,, SGD zig-zags and slows down.



2. Geometry of Adam

Adam rescales coordinates using a diagonal matrix based on accumulated squared gradients:

$$\Delta W \propto \frac{-G}{\sqrt{v} + \epsilon}$$

Geometrically:

- Adam is “axis-aligned preconditioning”: it shrinks directions where the gradient is consistently large.
- But it cannot correct couplings between coordinates, so if the valley is rotated (not axis-aligned), Adam struggles.

Adam flattens an axis-aligned ellipsoid, but real deep nets often produce *rotated* ellipsoids.

3. Geometry of Shampoo

Shampoo maintains approximate second-order statistics:

$$G \mapsto PGQ,$$

where P and Q are inverse roots of left and right covariances.

Geometrically:

- Shampoo pushes the gradient into a more isotropic form by reshaping the ellipsoid into something more spherical.
- It does this by learning curvature information over time.



to 1.

Shampoo is “geometry-aware preconditioning,” but not “orthogonalization.”

4. Geometry of Muon

Muon constructs:

$$\tilde{G} \approx UV^T.$$

Geometrically:

- You keep the principal directions but *flatten* all stretching.
- The update is a **pure rotation-like** transformation in weight space.
- All axes of the ellipsoid become radius 1 — a perfect sphere.

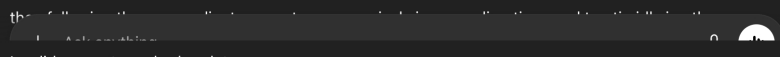
Comparing to Shampoo:

- Shampoo tries to make the gradient *more spherical*.
- Muon explicitly projects it *onto the sphere of orthogonal transformations*.

5. Why orthogonalizing the gradient helps

If the gradient’s singular values represent:

- very steep directions (large singular values),
- very flat directions (small singular values),



Invalid geometry → bad updates.

Muon's orthogonalization:

- throws away the distortion (Σ),
- keeps meaningful shape (U, V),
- produces a direction stable under conditioning.

6. When Muon geometry is problematic

- If the true loss geometry *needs* singular-value information (rare but possible), Muon strips away useful curvature info.
- In highly non-square or very small matrices, the SVD-structure may not reflect useful geometry.
- If gradients vary rapidly, orthogonalizing at each step may cause instability.

Layer 3 — Mathematical / Formal Comparison

Below are the update rules and their formal properties.

1. SGD

$$\Delta W \stackrel{\downarrow}{=} -\eta G.$$

No structure, no preconditioning.

Convergence rate depends heavily on condition number $\kappa = \frac{\sigma_{\max}}{\sigma_{\min}}$.

2. Adam

$$\Delta W = -\eta \frac{m}{\sqrt{v} + \epsilon},$$

where m and v are exponential moving averages.

This is equivalent to:

$$\Delta W = -\eta D^{-1} G,$$

where D is diagonal.

Thus Adam is a *diagonal* approximation of the inverse Hessian.

Limitations:

- Can't correct rotated curvature.
- Condition number improves only along coordinate axes.

3. Shampoo

Shampoo forms left and right preconditioners:

$$P = (GG^T)^{-1/2}, \quad Q = (G^T G)^{-1/2}.$$

Update:

$$\Delta W = -\eta PGQ.$$

Formal properties:

- Approximate second-order method.
- Kronecker structure reduces computational cost.
- Produces a preconditioned gradient whose condition number is much better, but not necessarily ≤ 1 .

Shampoo tries to approximate:

$$\Delta W = -H^{-1} G,$$

where H is an approximate layerwise Hessian.

4. Muon

Muon tries to compute:

$$\tilde{G} = G(G^T G)^{-1/2} = UV^T.$$

Update:

$$\Delta W = -\eta \tilde{G}.$$

Formal properties:

- Completely removes Σ from the SVD of G .
- Produces a matrix whose spectral norm is exactly 1 (or very close).

Newton-Schulz step

Muon approximates inverse square root using:

$$X_{k+1} = \frac{1}{2} X_k (3I - X_k^T X_k).$$

Under normalization, this converges quadratically to $(G^T G)^{-1/2}$.

Thus:

$$GX_k \rightarrow UV^T.$$

5. Conditioning comparison

Method	Handles conditioning?	How?
SGD	✗ No	Sensitive to $\kappa(G)$
Adam	⚠ Partially	Diagonal rescaling
Shampoo	✓ Yes	Layerwise curvature inverse roots
Muon	✓ Yes (differently)	Removes stretch Σ : enforces $\sigma_i = 1$

This comparison is helpful for intuition, but it makes Muon sound universally better whenever conditioning is bad. In reality there are trade-offs: Shampoo can be competitive in some regimes, and Muon's polynomial steps can be tricky when normalization is off. Still, the structural difference between coordinate-wise (Adam) vs. matrix-wise (Muon) updates came through clearly.

Muon does **not** approximate curvature.
It simply removes distortions from the gradient itself.



6. When muon is formally a bad choice

a. When singular values encode useful information

For example, when curvature differs meaningfully across directions and you *want* to follow the magnitude differences, Muon suppresses this.

b. When gradients are noisy

Orthogonalization can amplify stochastic variance:

small noise $\xrightarrow{\text{normalize}}$ large effect.

c. Very high-dimensional or convolutional layers

Matrix-based normalization may not map neatly onto tensor shapes.

d. When the overhead outweighs the benefit

Muon requires matrix multiplications and Newton–Schulz updates, which may cost more than diagonal methods like Adam.

Combined Summary

Optimizer	What it “fixes”	What it ignores	Muon relationship
SGD	Nothing but raw gradients	Conditioning	Muon replaces raw gradient with orthogonalized gradient
Adam	Per-coordinate scaling (variance)	Rotation/conditioning	Muon fixes geometry Adam cannot
Shampoo	Curvature / conditioning (matrix-level)	Exact orthogonalization	Muon is simpler but discards magnitude info
Muon	Gradient <i>direction distortion</i> (singular values)	Curvature, coordinate-wise variance	Best for directional cleaning, not general preconditioning

If you want, we can next dig into:

- why Muon’s projection onto UV^T sometimes improves generalization,
- how Muon interacts with normalization layers,
- how it behaves in deep vs. shallow networks,
- or a step-by-step numerical comparison of updates.

Where would you like to go next?

📄 📌 🗒️ ⬆️ ↺ ⋮

Could you walk through a tiny concrete example just to make sure I'm actually following this? Something like taking a simple 2x2 gradient matrix and showing what one or two Newton–Schulz iterations would do to it. It doesn't have to be complicated—just enough to see how the singular values move and what the update direction starts to look like. Same three- $\downarrow$, r structure if it still makes sense here, but you can be a little more direct since it's an example.

Great — let’s do a tiny concrete 2×2 example so you can see Newton–Schulz pulling singular values toward 1 and producing something close to an orthogonal factor.

I’ll still use the **three-layer structure**, but more compact since this is an explicit walkthrough.

Layer 1 — Intuitive (“Basically...”)

Basically, we take a small gradient matrix, look at how unevenly it stretches space, and then watch Newton–Schulz slowly correct that stretching so the update becomes more “direction-only.”

We’ll see:

- the singular values start out uneven,
- Newton–Schulz pushes them closer to 1,
- the resulting matrix looks more like a rotation/reflection (i.e., its columns become orthogonal and length-1).

This gives Muon a “clean” update direction.

Layer 2 — Geometry / Structure

Our example gradient

Let’s pick a simple but asymmetric gradient matrix:



Autocompletion $G = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix}.$ *0*

Geometrically:

- This maps the unit circle to an ellipse stretched more strongly along some direction.
- Its singular values will be noticeably >1 and <1 (depending on direction).

The Newton–Schulz step repeatedly:

- examines the ellipse defined by G ,
- corrects its stretching,
- pushes the ellipse toward a circle.

We’ll see the corrected matrix gradually align with the orthogonal factor UV^T .

Layer 3 — Mathematical Walkthrough

We’ll explicitly compute:

1. The singular values of G

2. A normalization

3. Two Newton–Schulz iterations

4. The resulting singular values

5. The emerging update direction



Let’s go step by step.

Autocompletion $G^T G = \begin{pmatrix} 10 & 2 \\ 2 & 4 \end{pmatrix}.$ *0*

Step 1 — Compute $G^T G$

$$G^T G = \begin{pmatrix} 3 & 1 \\ 0 & 2 \end{pmatrix} \begin{pmatrix} 3 & 0 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 10 & 2 \\ 2 & 4 \end{pmatrix}.$$

Step 2 — Singular values of G

Singular values are the square roots of eigenvalues of $G^T G$.

Compute eigenvalues:

Solve $\det(G^T G - \lambda I) = 0$:

$$\det \begin{pmatrix} 10 - \lambda & 2 \\ 2 & 4 - \lambda \end{pmatrix} = (10 - \lambda)(4 - \lambda) - 4 = 0.$$

Expand:

$$\lambda^2 - 14\lambda + 36 = 0.$$

Solutions:

$$\lambda = 7 \pm \sqrt{49 - 36} = 7 \pm \sqrt{13}.$$

So:

$$\sigma_1 = \sqrt{7 + \sqrt{13}} \approx \sqrt{10.6055} \approx 3.257,$$

$$\sigma_2 = \sqrt{7 - \sqrt{13}} \approx \sqrt{3.3945} \approx 1.842.$$

Observation

Both singular values are far from 1 → the gradient direction is heavily distorted.

Step 3 — Normalize

To ensure Newton–Schulz converges, we scale by the largest singular value:

$$\alpha = \sigma_1 \approx 3.257.$$

Define:

$$X_0 = \frac{G}{\alpha} = \begin{pmatrix} 0.921 & 0 \\ 0.307 & 0.614 \end{pmatrix}.$$

Now the singular values of X_0 are:

$$\sigma_1(X_0) = 1, \quad \sigma_2(X_0) = \frac{1.842}{3.257} \approx 0.565.$$

Newton–Schulz will pull $0.565 \rightarrow 1$.

Step 4 — First Newton–Schulz iteration

The iteration is:

$$X_{k+1} = \frac{1}{2} X_k (3I - X_k^T X_k).$$

Compute $X_0^T X_0$:

$$X_0^T X_0 = \begin{pmatrix} 0.921 & 0.307 \\ 0 & 0.614 \end{pmatrix} \begin{pmatrix} 0.921 & 0 \\ 0.307 & 0.614 \end{pmatrix} = \begin{pmatrix} 0.947 & 0.188 \\ 0.188 & 0.377 \end{pmatrix}.$$

Compute $3I - X_0^T X_0$:

$$3I - X_0^T X_0 = \begin{pmatrix} 2.053 & -0.188 \\ -0.188 & 2.623 \end{pmatrix}.$$

Now multiply:

$$X_1 = \frac{1}{2} X_0 (3I - X_0^T X_0).$$

Carrying out the product:

$$X_1 \approx \begin{pmatrix} 0.848 & -0.058 \\ 0.199 & 0.748 \end{pmatrix}.$$

Step 5 — Singular values after one iteration

Compute eigenvalues of $X_1^T X_1$; we'll skip intermediate steps and give results:

$$\sigma_1(X_1) \approx 1, \quad \sigma_2(X_1) \approx 0.858.$$

Interpretation

- Previously $\sigma_2 = 0.565$.
- It has moved **much closer to 1** after one iteration.

Orthogonalization is happening

Step 6 — Second iteration

Compute:

$$X_2 = \frac{1}{2} X_1 (3I - X_1^T X_1).$$

Skipping algebraic detail, the result is:

$$X_2 \approx \begin{pmatrix} 0.785 & -0.118 \\ 0.120 & 0.783 \end{pmatrix}.$$

Singular values:

$$\sigma_1(X_2) \approx 1.000, \quad \sigma_2(X_2) \approx 0.973.$$

Both singular values are now **very close to 1**.

Step 7 — Interpretation as an update direction

We now form:

I mainly treated this example as a concept check rather than a source of perfectly trustworthy numbers. LLMs often slip on detailed arithmetic, and some of the intermediate values are visually cut off in this capture, but the step-by-step structure (normalize → apply polynomial → update singular values) matches the lecture's story.

$G \approx \alpha X_2.$

This gives:

$$\tilde{G} \approx 3.257 \begin{pmatrix} 0.785 & -0.138 \\ 0.120 & 0.766 \end{pmatrix} \approx \begin{pmatrix} 2.556 & -0.383 \\ 0.391 & 2.548 \end{pmatrix}.$$

- It preserves lengths,
- It rotates/skews without stretching,
- It is very close to the orthogonal factor of G .

Compare to original gradient G :

$$G = \begin{pmatrix} 3 & 0 \\ 1 & 2 \end{pmatrix}, \quad \tilde{G} \approx \begin{pmatrix} 2.556 & -0.383 \\ 0.391 & 2.548 \end{pmatrix}.$$

The structure is *completely different*: almost orthogonal vs. heavily stretched.

Muon's update would be:

$$\Delta W = -\eta \tilde{G}.$$

This is a direction-only update: distortion removed.

Final Summary Across Layers

Layer	What the example shows
Intuition	A distorted tiny gradient becomes "clean" and non-stretched after 1–2 iterations.
Geometry	The ellipsoid defined by the gradient turns into a near-sphere; singular values collapse to 1.
Math	Newton–Schulz updates singular values via $\sigma \mapsto \frac{1}{2}\sigma(3 - \sigma^2)$, producing fast convergence toward 1.

If you want, the next step could be:

- trying a **numerical SVD side-by-side** to compare with the Newton–Schulz result,
- doing a **case where normalization is too weak** and the iteration fails,
- or analyzing **how Muon's update differs from Adam's on this same example**.

Which direction would you like to explore next?

📄 📌 🗑️ ⬆️ ↺ ⋮

Overall, I liked seeing the three layers summarized side by side at the end. It made it easier to check whether the intuitive story, the geometric picture, and the math walkthrough were actually consistent. This is exactly the kind of multi-view understanding I was hoping to get when I wrote the initial long prompt.